MANUELA ADELYNN SITOMPUL
2415455641 / TK 160698
Algoritma dan Struktur Data
C

Problem 1 : Algorithm Analysis

Snippet A

```
// n >= 1
int sum = 0 ;
for (int i = 0 ; i < n ; i++) {
    for (int j = 0 ; j < i ; j++) {
        sum += arr[i] + arr[j];
    }
}
```

a) Exact operation count T(n) ; rule : assignment, comparisons, arithmatic operation, array = cost 1

:

int sum = 0 : 1 assignment. cost 1, Execute : 1 times. Total = 1

Outer Loop for (int i = 0 ; i < n ; i ++)

initialing int i = 0 : 1 assignment cost 1 Execute : 1 time Total : 1

condition i < n : 1 comparison cost 1 Execute : n+1 times Total : n+1

update i++ : 2 arithmetic, assignment cost 2 Execute : n times Total : 2n

Inner Loop for (int j = 0 ; j < i ; j++)

initialing int j = 0 : 1 assignment cost 1 Execute : n times Total : n

condition j < i : 1 comparison cost 1 Execute : $\frac{n(n+2)}{2}$ times Total : $\frac{n^2}{n} + \frac{n}{2}$

update j++ : 2 arithmetic, assignment cost 2 Execute : $\frac{n(n-1)}{n}$ times Total : $n^2 - n$

sum += arr[i] + arr[j] : 5 (array access, arithmetic, assignment) cost 5 Execute : $\frac{n(n-1)}{2}$

Total : $\frac{5}{2}n^2 - \frac{5}{2}n$

$T(n) = n^2 = \frac{1}{2}n^2 + n^2 + \frac{5}{2}n^2$; $n = n + 2n + n + \frac{1}{2}n - n - \frac{5}{2}n$

$= 4n^2$ $= n$

constant = 3

$T(n) = 4n^2 + n + 3$

b) Big O-class (* ignore lower orderr, constant of the higher order)

: $T(n) = 4n^2 + n + 3$. We get $O(n^2)$

c) Best / Worst case

: They do not differ here, because there is no early exit and the loop bounds depend only on n, not on the array values. The program always performs the same number of loop iterations for a given n.

GELATIK

CS Dipindai dengan CamScanner

Snipet B

```
// n > 1 ; arr is a sorted array of n integer
int lo = 0, hi = n-1, target = 18;
bool found = false
while (lo <= hi) {
    int mid = lo + (hi-lo)/2 ;
    if (arr [mid] == target)    { found = true ; break; }
    else if (arr [mid] < target)  { lo = mid + 1 ; }
    else                        { hi = mid - 1 ; }
}
```

a) Exact operation count T(n)

: Just assume we count in the worst case. Then, each iteration will be $\frac{1}{2}$ (half) the search space, and the maximum loop iteration while would be $a = \lfloor \log_2 n \rfloor + 1$

1), 3) Initializing

cost = 5 (4 assignment, 1 arithmetic)

Execute: 1 Total: 5

4) while (lo <= hi)

cost = 1 (comparison)

Execute: a+1 Total: a+1

5) mid = lo + (hi-lo)/2

cost = 4 (1 assignment, 3 arithmetic)

Execute: a Total: 4a

6) if (arr [mid] == target)

cost = 2 (array access, comparison)

Execute: a Total: 2a

7) else if (arr [mid] < target)

cost = 2 (array access, comparison)

Execute: a Total: 2a

8) else { hi = mid - 1; }

cost = 2 (assignment, arithmetic)

Execute: a Total: 2a

: $T(n) = 5 + (a+1) + 4a + 2a + 2a + 2a$

$= 11a + 6$

: $a = \log_2 n$

: $T(n) = 11(\log_2 n) + 6$

$= 11 \log_2 n + 11 + 6$

$= 11 \log_2 n + 17$

Dipindai dengan CamScanner

b) Big O-class

: Since the number of iterations of the loop increases logarithmically with the size of the array, the overall time complexity of the algorithm is $O(\log n)$

c) Best/worst case

: Best case = Happens when the target value is located exactly at the middle position during the first iteration. In this situation, the algorithm immidiately finds the element and exits the loop. Time Complexity: $O(1)$

: Worst case = Happens when the target element is either not present in the array or is found only after repeatedly halving the search space until only one element remains. In this case, the loop runs the maximum number of iterations : $\lfloor \log_2 n \rfloor + 1$. Thus the worst case complexity: $O(\log n)$

Snippet C

```
// n >= 1
int count = 0;
for (int i=0; i<n; i++){
    for (int j=i+1; j<n; j++) {
        for (int k=0; k<n; k+=2) {
            if (arr[i] != arr[j] && arr[k] > 0)
                count ++;
        }
    }
}
```

a) Exact operation count $T(n)$

The total number of executions of the innermost block depends on the number of iterations of the loops.

- the two outer loops (i and j) generate all unique index pairs. The total number of such pairs is: $L_2 = \frac{n(n-1)}{2}$
- the innermost loop (k) increases by 2 each iteration, so it runs approximately: $M = \lfloor \frac{n}{2} \rfloor$ times per (i,j) pair.

Assuming the worst case, the condition inside the loop is always true and count ++ executes every time.

2) cost : 1 (assignment) Execute: 1 Total: 1

3) int i=0; cost : 1 (assignment) Execute: 1 Total: 1

i<n; cost : 1 (comparison) Execute: $n+1$ Total: $n+1$

i++; cost : 2 (assignment, arithmetic) Execute: n Total: $2n$

4) int j = i+1; cost : 2 (assignment, arithmetic) Execute: n Total: $2n$

j<n; cost : 1 (comparison) Execute: $\frac{n(n+1)}{2}$ Total: $\frac{n^2}{2} + \frac{n}{2}$

j++; cost : 2 (assignment, arithmetic) Execute: $\frac{n^2-n}{2}$ Total: n^2-n

5) int k=0; cost : 1 (assignment) Execute: L_2 Total: $\frac{1}{2}n^2 - \frac{n}{2}$

k<n; cost : 1 (comparison) Execute: $L_2(M+1)$ Total: $L_2(M+1)$

k+=2; cost : 2 (assignment, arithmetic) Execute: $L_2 \times M$ Total: $2L_2 \times M$

6) if (arr[i] != arr[j] && arr[k] > 0); cost : 6 (3 array, 2 compare, 1 logic) Execute: $L_2 M$ Total: $6L_2M$

7) count ++; cost : 2 (assignment, arithmetic) Execute: $L_2 \times M$ Total: $2L_2 \times M$

GELATIK

Dipindai dengan CamScanner

: $T(n) = 1 + (3n+2) + (\frac{3}{2}n^2 + \frac{3}{2}n) + (\frac{3}{4}n^3 + \frac{1}{4}n^2 - n) + (\frac{3}{2}n^3 - \frac{3}{2}n^2) + (\frac{1}{2}n^3 - \frac{1}{2}n^2)$

$= \frac{11}{4}n^3 - \frac{1}{4}n^2 + \frac{7}{2}n + 3$

b) Big O-class

: When determining asymptotic complexity, only the highest-degree term matters. The dominant term in $T(n)$ is: $\frac{11}{4}n^3$. Therefore, the time complexity is : $O(n^3)$.

c) Best/worst case (* condition inside the loop: if (arr[i] != arr[j] && arr[k] > 0)

: Because the loop always iterate the same number of times regardless of the data values, the algorithm performs roughly the same number of iterations in all cases. Even though the number of count ++ operation differs, the loop structure doesn't change. Therefore, both case have the same complexity: $O(n^3)$.

Problem 3 : B

Complexity Comparison Table

Operation	Best Case	Worst Case	Average Case
① Linear Search (unsorted)	$O(1)$	$O(n)$	$O(n)$
② Linear Search (sorted, early exit)	$O(1)$	$O(n)$	$O(n)$
③ Binary Search (sorted)	$O(1)$	$O(\log n)$	$O(\log n)$

: ① Linear Search (unsorted array): the algorithm checks elements sequentially from the begining until the target is found or the array ends. In the worst situation, every element must be examined.

Best Case : $O(1)$ — the target is located at the first index, so only one comparison is required

Worst Case : $O(n)$ — the target is either not in the array or appears at the last position, forcing the algorithm to inspect all n elements.

Avg. Case : $O(n)$ — on averge, the algorithm checks about $\frac{n}{2}$ elements, which still grows linearly with n.

② Linear Search (sorted, early exit): the array is sorted, so the search can stop early when it encounters a value greater than the target

Best Case : $O(1)$ — the target is at the first element or the first element already exceeds the target

Worst Case : $O(n)$ — the target is larger than all elements or located at the end, so every element must still be checked

Avg. Case : $O(n)$ — although early termination may happen, the algorithm still scans a linear portion of the array on average

③ Binary Search (sorted array): repeatedly halves the search interval by comparing the target with the middle element.

Best Case : the target matches the middle element during the first comparison.

Worst Case : the target isn't present or lies at an extreme end, so the search continues until the interval size become 1

Avg. Case : most element are found within approximately $\log_2 n$ iterations because the search space is halved each step

: Reasoning : binary become more efficient than linear search when the array size reaches $n = 3$

example : $n = 1$ — linear = 1 comparison, binary = 1 comparison

$n = 2$ — both require about 2 comparisons

$n = 3$ — binary search requires about 2 iterations, while linear search may require 3 checks.

Therefore, starting from $n = 3$, binary search performs fewer comparisons than linear search in the worst case.

Dipindai dengan CamScanner

Problem 4 : Amortized Analysis of pushBack

A.

Call #	Value Pushed	size after		capacity after		Cost	
1	10	1	1	2	2	1	1
2	20	2	2	2	2	1	1
3	30	3	3	4	6	3	3
4	40	4	4	4	6	1	1
5	50	5	5	8	6	5	1
6	60	6	6	8	10	1	1
7	70	7	7	8	10	1	7
8	80	8	8	8	10	1	1
9	80	9	9	16	10	9	1
10	100	10	10	16	10	1	1
11	110	11	11	16	14	1	11
12	120	12	12	16	14	1	1

B. Total Cost

1. Sum all values in the Cost column : $T_{12} = 1+1+3+1+5+1+1+1+9+1+1+1 = 26$

2. Average cost per call, i.e., $T_{12}/12$: Average $= \frac{26}{12} \approx 2.7$ operations

3. Total copy work (geometric series); resizing requires copying the existing elements. The number of copied elements follows a geometric sequence: $2+4+8+\dots+2^k$, where 2^k is the largest power of ~~two~~ smaller than n. The sum of this geometric series is: $2^{k+1}-2$. Since $2^k < n$, the total copying work is less than $2n$.

4. Final amortized result : $T(n) = n$ (normal appends) $+ O(n)$ (copies). Thus, $T(n) = O(n)$, and the amortized cost per pushBack is $= \frac{T(n)}{n} = O(1)$

C. Growth Factor Comparison

1. (* the new ones are on the table up the, on the right side)

2. Compare the total cost of both strategies.

total cost with doubling strategy : 26
total cost with fixed increment: 36

} this shows that doubling strategy is already more efficient even for small input sizes.

3. General derivations : Doubling strategy: $T(n) = n$ appends $+ O(n)$ copies $= O(n)$. Amortized cost per operation: $O(1)$

Fixed increment strategy: Resizing happens approximately $\frac{n}{k}$ times, and the copied elements grow linearly each time. The total copying work forms an arithmetic series: $O(n^2)$. So, $T(n) = O(n^2)$. Amortized cost per operation: $O(n)$

GELATIK

Dipindai dengan CamScanner